

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Created on Sun Nov 3 16:29:08 2024

@author: sebastiencanard
"""

import random
import hashlib
import secrets
from sympy import isprime

"""
Génération des nombres premiers et des clés RSA
"""

def generate_prime(bits):
    """Génère un nombre premier de taille spécifiée en bits."""
    while True:
        num = random.getrandbits(bits)
        if isprime(num):
            return num

def generate_keys(bits):
    """Génère des clés RSA (public et privé) de taille spécifiée en bits."""
    """L'entrée "bit" donne la taille du module n"""

    # Génération de p et q (grands nombres premiers)
    p = generate_prime(bits//2)
    q = generate_prime(bits//2)
    while p == q:
        q = generate_prime(bits//2)

    # Calcul de n et  $\phi(n)$ 
    n = p * q
    phi_n = (p - 1) * (q - 1)

    # Choix aléatoire de e
    e = generate_prime(bits)
    #e = generate_prime(bits*2) Version avec génération aléatoire
    # Calcul de d comme inverse modulaire de e modulo  $\phi(n)$ 
    d = pow(e, -1, phi_n)

    # Clé publique (n, e) et clé privée (n, d)
    return (n, e), (n, d)

def bytes_to_int(byte_list):
    return int.from_bytes(bytes(byte_list), byteorder='big', signed=False)

def int_to_bytes(entier, length):
    return list(int.to_bytes(entier, length=length, byteorder='big', signed=False))

"""
Question 2.1 - Chiffrement et déchiffrement RSA de base
"""

def encrypt_rsa(message, public_key):
    """Chiffre un message avec RSA et la clé publique (n, e)."""
    n, e = public_key
    message_int = bytes_to_int(message)
    cipher_int = pow(message_int, e, n)
    return cipher_int

def decrypt_rsa(cipher_int, private_key):
    n, d = private_key
    mod_bytes = (n.bit_length() + 7) // 8
    message_int = pow(cipher_int, d, n)
    return int_to_bytes(message_int, mod_bytes)
```

```
"""
```

Question 2.2 - Implémentation de RSA-OAEP

```
"""
```

```
def pkcs7_pad(data, block_size=32):
    """
    Pads the given plaintext with PKCS#7 padding to a multiple of 16 bytes.
    Note that if the plaintext size is a multiple of 16,
    a whole block will be added.
    """
    padding_len = block_size - (len(data) % block_size)
    padding = [padding_len] * padding_len
    return data + padding

def pkcs7_unpad(data, block_size=32):
    """
    Removes a PKCS#7 padding, returning the unpadded text and ensuring the
    padding was correct.
    """
    padding_len = data[-1]
    return data[:-padding_len]

def hash_data(data, hash_algo=hashlib.sha256):
    return hash_algo(data).digest()

def get_hash_length(hash_algo=hashlib.sha256):
    return hash_algo().digest_size

def oaep_encode(message, hash_algo=hashlib.sha256):
    h_len = get_hash_length(hash_algo)

    message = pkcs7_pad(message)

    seed = secrets.token_bytes(h_len)

    # Assurez-vous que s_mask a la même longueur que message
    s_mask = hash_data(seed, hash_algo)
    s = bytes([message[i] ^ s_mask[i % len(s_mask)] for i in range(h_len)])

    # Assurez-vous que t_mask a la même longueur que seed
    t_mask = hash_data(s, hash_algo)
    t = bytes([seed[i] ^ t_mask[i % len(t_mask)] for i in range(h_len)])

    encoded = t + s

    return encoded

def oaep_decode(encoded, hash_algo=hashlib.sha256):
    h_len = get_hash_length(hash_algo)

    size_encoded = len(encoded)

    # Récupère t et s selon leur taille.
    t = encoded[size_encoded-2*h_len: size_encoded-h_len]
    s = encoded[size_encoded-h_len: size_encoded]

    # Assurez-vous que seed_mask a la même longueur que t
    seed_mask = hash_data(bytes(s), hash_algo)
    seed = bytes([t[i] ^ seed_mask[i % len(seed_mask)] for i in range(h_len)])

    # Assurez-vous que m_mask a la même longueur que s
    m_mask = hash_data(seed, hash_algo)
    message = bytes([s[i] ^ m_mask[i % len(m_mask)] for i in range(len(s))])

    return pkcs7_unpad(message)
```

```
def encrypt_rsa_oaep(message, public_key):
    padded_message = oaep_encode(message)
    cipher = encrypt_rsa(padded_message, public_key)
    return cipher

def decrypt_rsa_oaep(cipher, private_key):
    padded_message = decrypt_rsa(cipher, private_key)
    message = oaep_decode(padded_message)
    return list(message)
```

```
"""
```

```
Réponses aux questions de la section 2.2
```

```
"""
```

```
print("-----")
print("Question 2.2 - RSA-OAEP")
print("-----")
```

```
""" Génération des clés RSA (2048 bits) """
public_key, private_key = generate_keys(bits=2048)
n, e = public_key
```

```
""" Message à chiffrer """
message = [0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae, 0xd2, 0xa6, 0xab, 0xf7, 0xcf, 0x80, 0x09, 0x89,
0x01, 0x0c]
print("Message avant chiffrement :", message, "\n")
```

```
""" Chiffrement """
cipher_int = encrypt_rsa_oaep(message, public_key)
```

```
""" Déchiffrement """
plaintext = decrypt_rsa_oaep(cipher_int, private_key)
print("Message déchiffré :", list(plaintext), "\n")
```